

UNIT – V

Agile Design Practices

Embrace Flexibility, Not Rigidity: Agile Design in Action

1. Adaptability: The Core of Agile Design

Unlike traditional methods that get bogged down in upfront planning, Agile design thrives on adaptability. It's a toolbox of practices that lets the design evolve organically as the project progresses, constantly informed by user needs, development progress, and overall feedback.

2. Planning with Flexibility: A Blueprint, Not a Cage

This doesn't mean there's no plan – Agile teams start with a basic blueprint to guide them. But the key is staying flexible. Think of it like building a house: you have a foundation, but you can adjust the layout as you go based on what works best.

3. Tools in the Agile Toolbox: Test-Driven Development and Beyond

One powerful Agile practice is [Test-Driven Development \(TDD\)](#). Here, developers write tests before the actual code, ensuring high quality and early feedback. But TDD is just one tool in the Agile belt. Agile values flexibility, so the specific practices used depend on your project and team.

By embracing this adaptability, Agile design empowers teams to respond quickly to changes, minimize technical debt, and ultimately deliver exceptional user experiences

Agile Automated Tools

Tools help teams with planning, visualization, collaboration, process streamlining, and maintaining focus throughout the entire project journey.

1. Best for issue tracking and sprint planning: Jira

[Jira](#) is purpose-built for software teams, who rely on agile ways of working. It keeps teams focused, motivated, and on schedule by breaking large, complex projects into smaller, more

manageable sprints.

As a sprint planning tool, it supports refining [product backlogs](#), generating and prioritizing ideas, documenting user stories, and defining the tasks required to achieve a successful release. Project managers can create a continuous delivery pipeline with [agile project plan templates](#), as well as templates for agile collaboration at scale. And, it's free for up to 10 users.

2. Best for aligning with cross-functional teams: Jira

Coordinating projects that involve multiple teams, such as software development and marketing, can be a complex task. Each team has its own way of working based on its specific discipline and focus. When non- software teams adopt agile project management methods, it's especially important to recognize and address the differences in how each team works.

With [Jira](#), agile project management is synchronized across the organization to encompass all resources and departments involved. [Work management solutions](#) allow cross-business teams to identify dependencies between business areas, manage resource allocation, and track tasks related to sprints, and even epics, in one visual workflow.

3. Best for sprint retrospectives: Confluence

Sprint retrospectives are one of the most important continuous improvement tools available to an agile development team. They provide an opportunity for everyone to share what went well and where there's room for improvement. The most successful teams look beyond a single sprint and instead, view trends across multiple projects. An anomaly observed in one sprint may actually be a recurring issue that becomes more apparent when examined across projects.

[Confluence](#) provides a real-time, centralized knowledge management platform for collaborating on everything from customer feedback to sprint retrospectives. With the [sprint retrospective template](#), Confluence gives agile project teams the tools to capture important feedback for both immediate and future improvement.

4. Best for backlog management: Jira

Ongoing agile project management involves effectively managing the product backlog. This involves organizing work into logical groups, prioritizing tasks based on the project roadmap, and documenting requirements to ensure the team understands what to build and why. This is often referred to as [Scrum backlog refinement](#).

Think of Agile as the overarching methodology guiding your work and [Scrum](#) as the structured framework of activities that facilitates its successful implementation. Jira includes [Scrum templates](#) for [backlog refinement](#), [task prioritization](#), [dependency mapping](#), and [resource optimization](#), making backlog management easier. When the team is working from a well-organized and prioritized backlog, sprint planning is easier and delivery is faster.

5. Best for project tracking and reporting: Jira

When it comes to [real-time insights](#), Jira provides what you need when you need it. During

sprint planning, insights into the backlog help teams plan smarter sprints. During the development process, [agile metrics](#), dashboards, and insights are available right on the board, giving teams the information they need to make real-time, data-driven decisions. Then, development frequency and cycle time reports help optimize the delivery pipeline.

6. Best for daily standups: Zoom

Today's teams are often highly dispersed. For agile teams who use a [scrum standup](#) as a daily check-in to discuss the sprint progress, address roadblocks, and collaborate, this can present a significant challenge.

[Zoom](#) video hosting brings the team together as if they were all in the same room. Screen sharing allows the team to discuss the project board or capture decisions, the recording feature allows absent team members to catch up on discussions at a later time, and the chat function allows for the real-time sharing of links and other essential information.

7. Best for collaboration and knowledge management: Confluence

Agile projects include their own internal documentation, which team members use to meet the sprint goals. Whether it's capturing [user stories](#), [defining requirements](#) and [product specifications](#), or external information like product certifications, information must be linked to the project, readily available, and always discoverable by all stakeholders.

[Confluence](#) is a central repository that brings knowledge management into the [product development lifecycle](#). It houses relevant project information and supports easy collaboration. Contributors can add text, images, code, and tables and attach files for true collaboration.

8. Best for task and workflow automation: Jira

[Jira](#) is an effective tool for task management because it provides real-time visual status updates so agile teams can respond to changes as they arise. [Jira automation](#) lets people focus on what's important by automating repetitive tasks. Anyone on the agile team can sync tasks across projects and products because it doesn't require special coding. [Jira templates](#) make it easy to get started quickly, and Jira automation integrates with other commonly used tools such as Slack, [Open Devops](#), [Bitbucket](#), GitHub, and more.

9. Best for async video collaboration: Loom

Task switching and interruptions slow down work. Informing team members about project status, roadmap updates, or new strategies doesn't always require immediate disruptions. Asynchronous communication allows team members to engage with new information when it best suits their workflow.

[Loom](#) lets agile project managers turn statuses and other information into videos, allowing team members to view them at their own convenience. This tool is especially helpful for geographically dispersed teams to remain informed and included, no matter what time zone they are in.

Refactoring Techniques:

There are multiple refactoring techniques available that will make the existing code better in terms of performance and maintainability. The basic purpose of refactoring in Agile or any other methodology is “Leave a module in a better state than you found it”

The Refactoring techniques are categorized by Martin Fowler as follows:

- Composing methods
- Moving features within Objects
- Organizing data
- Simplifying Conditional Expression
- Making method calls simpler

Every category has certain refactoring techniques. Few techniques are explained below.

1. Composing Methods: It deals with the proper packaging of method or function code. Large methods/functions having very long complex logic implementation can reduce the maintainability and readability of the code. Applying refactoring techniques like extract methods will help make the code more maintainable and readable. A few of the refactoring techniques in composing method/function are detailed below:

- Extract Method
- Inline Method
- Replace method with method object

Extract Method: This is one of the popularly used Refactoring techniques. The implementation behind this technique is very simple. It consists of breaking up long methods by shifting complex chunks of code into new methods which have very descriptive identifiers. This method is used when:

- The code consists of long methods covering multiple logical flows that can be broken up into smaller methods
- If the same code is duplicated in more than one flow, move this to a single method and call from all other flows

Example:

Before Refactoring:

```
void printStudentRecord() {  
  
    printSchoolName();  
  
    // print details  
    System.out.println("Id: " + _id);  
}
```

```
System.out.println("name " + _name);  
  
}
```

After Refactoring:

```
void printStudentRecord() {  
    printSchoolName();  
    printStudentDetails();  
}  
  
void printStudentDetails() {  
    // print details  
    System.out.println("Id: " + _id);  
    System.out.println("name " + _name);  
}
```

The advantages of this method are:

- Proper code reorganization by grouping a set of statements into the single smaller method
- Reduces code duplication
- Increases the readability of the code

Inline method: In contrast to the Extract Method technique of refactoring, Inline method techniques suggests replacing a method/function call with the body of the method/function. It is suggested when the source code of the method/function is very small. This method is used when:

- When a function call causes a performance bottleneck
- When inline code increases the readability of the code since the body of the function is the same as the function name
- When there are too many delegations in the code

Example:

Before Refactoring

```
int getRating() {  
  
    return (moreThanFiveLateDeliveries()) ? 2 : 1;  
  
}
```

```
boolean moreThanFiveLateDeliveries() {  
  
    return _numberOfLateDeliveries > 5;  
}
```

```
}
```

After Refactoring

```
int getRating() {  
  
    return (_numberOfLateDeliveries > 5) ? 2 : 1;  
  
}
```

The advantages of this method are:

- Unnecessary complexity reduced, reduction in method calling overhead
- Increases the readability of the code

Replace method with method object: The Thumb rule of good code is that, it should be readable and easily manageable. At times we might come across a very long method having complicated logic implementation done inside it with many local variables. Such a method cannot be simplified by applying the extract method technique because of the usage of too many local variables, because passing around that many variables would be just as messy as the long method itself. In such cases, a class can be created for such a method. This class will have all local variables of that long method as its data members. One of the methods will be the long method. The complicated logic of this method can easily be simplified by applying the extract methods technique.

Example:

Before Refactoring

```
class Employee {  
  
    private:  
  
    // some declarations  
    double CalSalary(float fVCPI, float fCPI, float fPPF) {  
  
        double dIncomeTax;  
  
        double dHRA;  
  
        double dLTA;  
  
        // complicated salary calculations
```

```
}
```

```
};
```

After Refactoring

```
class SalaryCalculator {
```

```
private: float m_fVCPI;
```

```
float m_fCPI;
```

```
float m_fPPF;
```

```
double m_dIncomeTax;
```

```
double m_dHRA;
```

```
double m_dLTA;
```

```
public:
```

```
SalaryCalculator(float fVCPIpercent, float fCPIpercent, float  
fPF, double dIncomeTaxPercent, double dHRA, double dLTA) {
```

```
    // initialize private data members  
}
```

```
double CalculateSal() {
```

```
    // complicated salary calculations  
} };
```

```
class Employee {
```

```
private:
```

```
// some declarations
public:

double CalSalary() {

    SalaryCalculator obj(14, 13, 200.43, dITPercent, dHRA, dLTA);

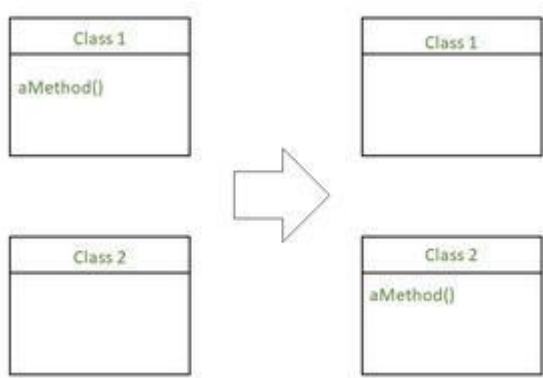
    return obj.CalculateSal();

};
```

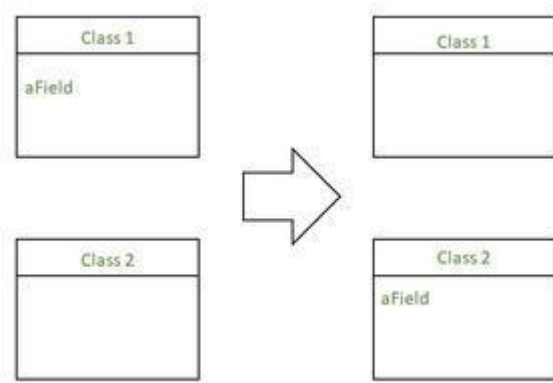
2. Moving Features Within Objects:

One of the most important and critical parts of object design is deciding on where to assign the responsibilities. There are many options. Refactoring techniques can help us decide the same. Following are a few of the refactoring techniques which can be used to decide the proper responsibility allocation:

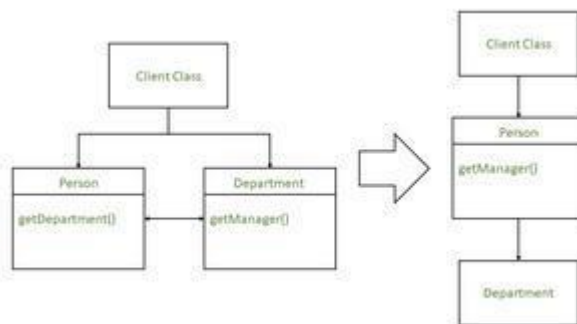
- **Move Method:** Consider a scenario for performing a functionality. A method is using members (data or functions) of another class i.e. Class-2 multiple times, as compared to the one in which it is written i.e. Class-1. Then such a code can be refactored using the Move Method technique. According to this technique in such cases, that method can be moved to Class-2. Class-1 can call this method in order to complete the needful functionality and the method can be removed from Class-1 altogether. Move method technique can also be applied when a method is used a maximum number of times by one of the classes in which it is declared and defined.



- **Move field:** If a member of one class is used a maximum number of times by access methods from some other class, then such field can be moved to another class itself. This needs necessary modifications in the earlier class which was having that data member in it. Move method techniques will also be applied when new classes are identified during the development phase.



- Hide delegate:** One of the important features of object-oriented design is if changes are made in one of the classes, then no other classes or very few classes should have an impact on its code because of this change. If one class is sending its requests to another class then such a class can be referred to as a client class. The class which serves the sent requests can be referred to as the server class. Consider a function that has to get an object of class B by calling one of the methods of class A. This function has to also call a method of class B to get the required data. This means that function has to interact with class A first and then with class B. In this case, the internal structure of class B is partially revealed to that function. The same can be avoided if class A has a method that itself will get the necessary things from class B and send the same to the client function. This delegation part can be made hidden from the client function. The below diagram shows a pictorial presentation where the 'Department' Class is hidden from 'Client', as the call to 'Department' Class is moved to the 'Person' class method.



3. Organizing Data:

The following section illustrates a few techniques that would make data organization better. This also makes working with data easier. Replace array with Object: One of the most common data structures used for data organization is an array. The array is a collection of homogeneous data in contiguous locations of memory having the same name. Although homogeneous, the same array can contain data with a different meaning causing confusion while accessing this data structure. In such cases, it is good to replace the array with an object.

4. Simplifying Conditional Expressions:

Understanding conditional logic implemented in a program can be one of the challenging tasks. Using refactoring techniques, the conditional logic can be simplified so that the code can be understood easily. This section has a few refactoring techniques using which conditional logic can be simplified easily.

5. Making method calls simpler:

Objects can interact with each other and the outside world through public methods. These methods should be easy to understand then the entire object-oriented design will become easy to understand. In this section, we will discuss a few refactoring techniques which will make the methods calls easier.

Agile Design Principles

Here are a few fundamental principles to implement the agile design process seamlessly:

1. Support from executives
2. Cross-functional teams
3. Project backlog and planning management
4. Accurate timelines for the next release
5. Research and testing
6. Minimizing team changes
7. Hiring a scrum master
8. Consistent communication

1. Support from Executives

Top management controls the budget and timeline, and their lack of support can lead to gaps in the project. Without executive backing, team members end up working with unrealistic timelines and insufficient resources, which may lead to overlooking essential aspects. It's crucial for teams to be granted, enough time and resources to ensure success.

A seamless product is only possible when executives understand the role design plays in a product and are willing to support the designers throughout the process.

2. Cross-functional Teams

The design teams should work with other team members, such as developers, marketing, etc., throughout the product development cycle. Connecting with relevant groups helps build trust among the team members and gauge the exact problems to create targeted solutions.

For example, developers and designers must consistently work together as they have common goals. When designers hand over the assets to developers, there's a risk that designs might be coded differently due to miscommunication or misunderstanding.

To avoid this, teams can create an introductory and interactive model instead of just sending across static designs. This ensures clear communication and helps developers better understand the intended functionality of design elements.

3. Project Backlog and Planning Management

Designers need to carefully prioritize and evaluate the backlog according to the value they provide, ensuring it contains features essential to the final product.

A user persona, empathy map, user experience (UX) storyboard, and analysis of the user journey can help significantly. These UX strategies can help in clearly identifying the user's needs.

4. Accurate Timelines for the Next Release

Besides highlighting features, designers should also highlight the time required to get the product ready for the market. Apart from providing a bird's eye view, estimations can also help manage the team activities in detail, allocating resources, setting realistic timelines, etc. Since agile methodology considers specific iterations, estimating the timings for those minor changes is vital too. You should also identify the complex and tedious tasks involved in the process so that enough time and resources are set aside to sort them out.

5. Research and Testing

Building a perfect product on time and within the budget means little if it doesn't align with what the users need. Ensuring the team is building the right solution for users is where research and testing can help.

When the team is stuck with minor changes, they may lose sight of the bigger picture. Products developed in isolation often fail to engage users. This may lead to a fragmented approach. Research and testing results can help the team stick to the end goal.

User research and testing are essential tools in the agile design process. This allows the team to avoid wasting time on redundant activities and ensure the product delivers the value users seek. Other ways to gather insights include conducting user interviews and field research.

6. Minimizing Team Changes

Teams thrive on positive team dynamics and strong team spirit. A new change can disrupt internal processes, directly impacting the team's performance. When new team members join, they need time to acclimate and familiarize themselves with existing workflows, which can slow down progress.

GET PROPOSALS

7. Hiring a Scrum Master

Introducing scrum processes to the team requires dedicated and experienced personnel. A professional scrum master ensures that the process is smooth. They act as a coordinator for agile product design activities and help the methodology integrate well within the team.

8. Consistent Communication

Communication is a critical aspect of any project, and it serves the same purpose within an agile team. When everyone is informed about ongoing activities, transparency is maintained, and no one works in isolation.

Apart from the team, the stakeholders and clients also benefit from consistent communication. Some strategies to do this are daily syncs, review sessions, and retrospective meetings, among others.

What is TDD?

Test-Driven Development (TDD) is a software development approach where tests are written before the code itself, following a cycle of Red-Green-Refactor. This means you write a test that will initially fail, then write code to make the test pass, and finally refactor the code to improve its design and structure.

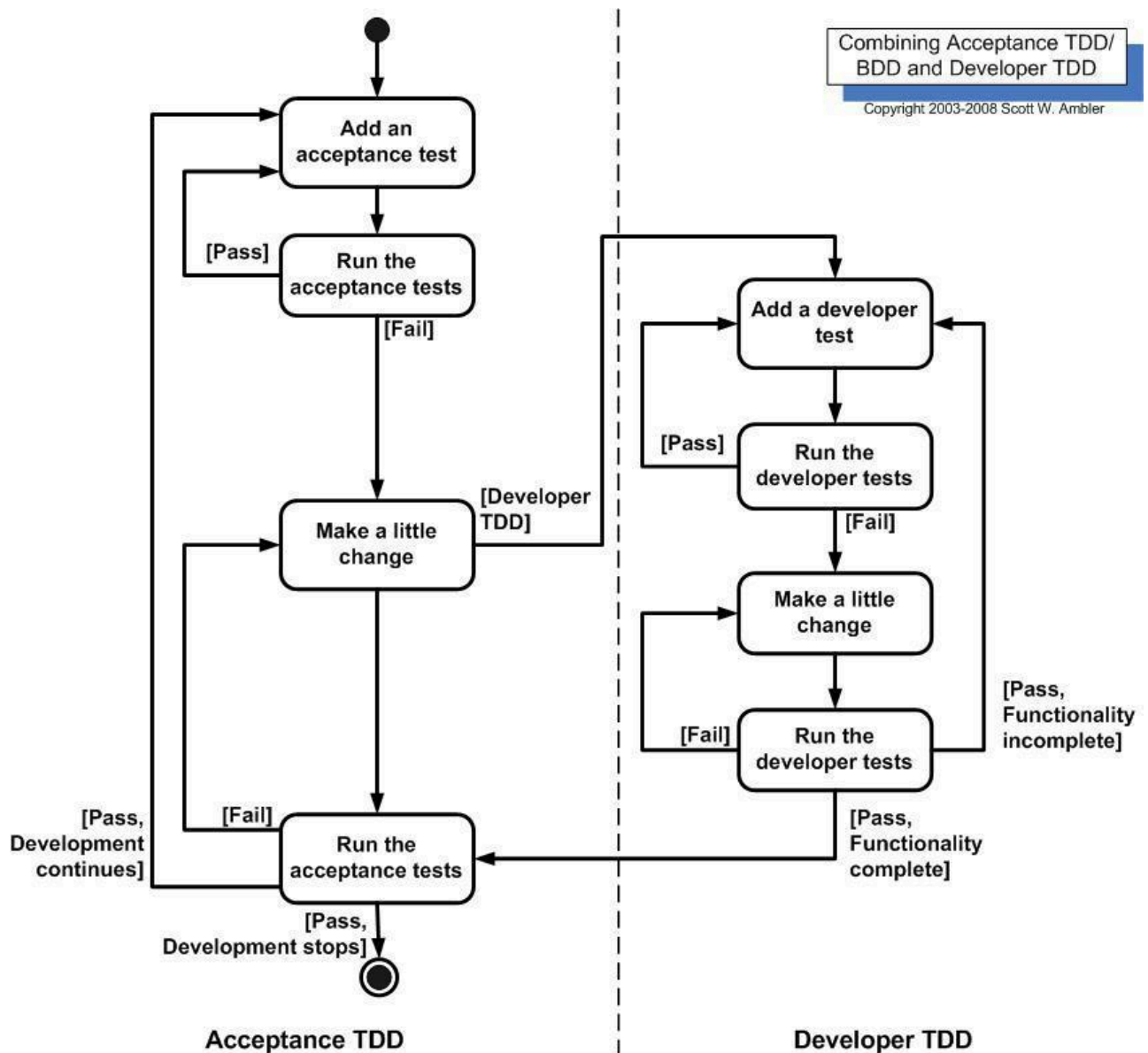
TDD with this simple formula: $TDD = Refactoring + TFD$. TDD completely turns traditional development around. A programmer taking a TDD approach refuses to write a new function until there is first a test that fails because that function isn't present. In fact, they refuse to add even a single line of code until a test exists for it. Once the test is in place they then do the work required to ensure that the test suite now passes (your new code may break several existing tests as well as the new one). This sounds simple in principle, but when you are first learning to take a TDD approach it proves require great discipline because it is easy to "slip" and write functional code without first writing a new test. One of the advantages of pair programming is that your pair helps you to stay on track. There are two levels of TDD:

Acceptance TDD (ATDD). With ATDD you write a single acceptance test or behavioural specification depending on your preferred terminology, and then just enough production functionality/code to fulfill that test. The goal of ATDD is to specify detailed, executable requirements for your solution on a just in time (JIT) basis. ATDD is also called Behavior Driven Development (BDD).

Developer TDD. With developer TDD you write a single developer test, sometimes inaccurately referred to as a unit test, and then just enough production code to fulfill that test. The goal of developer TDD is to specify a detailed, executable design for your solution on a JIT basis. Developer TDD is often simply called TDD.

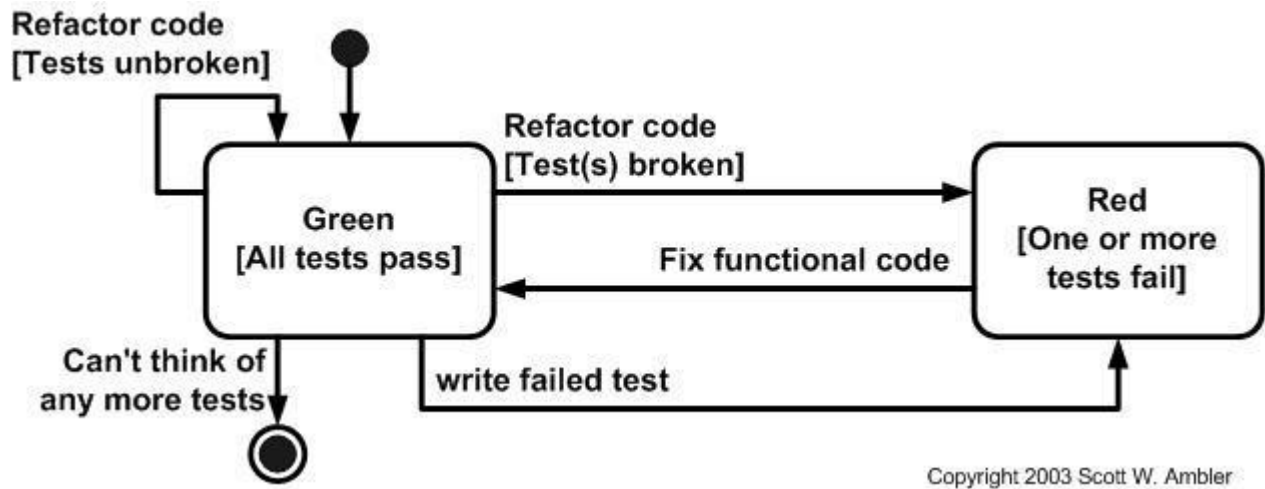
Figure depicts a UML activity diagram showing how ATDD and developer TDD fit together. Ideally, you'll write a single acceptance test, then to implement the production code required to fulfill that test you'll take a developer TDD approach. This in turn requires you to iterate several times through the write a test, write production code, get it working cycle at the developer TDD level.

Figure . How acceptance TDD and developer TDD work together.



Note that Figure assumes that you're doing both, although it is possible to do either one without the other. In fact, some teams will do developer TDD without doing ATDD, see survey results below, although if you're doing ATDD then it's pretty much certain you're also doing developer TDD. The challenge is that both forms of TDD require practitioners to have technical testing skills, skills that many requirement professionals often don't have (yet another reason why generalizing specialists are preferable to specialists). An underlying assumption of TDD is that you have a testing framework available to you. For acceptance TDD people will use tools such as Fitnesse or RSpec and for developer TDD agile software developers often use the xUnit family of open source tools, such as JUnit or VUnit, although commercial tools are also viable options. Without such tools TDD is virtually impossible. Figure 1 presents a UML state chart diagram for how people typically work with

such tools. This diagram was suggested to me by Keith Ray.
Figure 1. Testing via the xUnit Framework.



Kent Beck, who popularized TDD in eXtreme Programming (XP) (Beck 2000), defines two simple rules for TDD (Beck 2003). First, you should write new business code only when an automated test has failed. Second, you should eliminate any duplication that you find. Beck explains how these two simple rules generate complex individual and group behaviour:

Develop organically, with the running code providing feedback between decisions.

Write your own tests because you can't wait 20 times per day for someone else to write them for you.

Development environment must provide rapid response to small changes (e.g you need a fast compiler and regression test suite).

Designs must consist of highly cohesive, loosely coupled components (e.g. your design is highly normalized) to make testing easier (this also makes evolution and maintenance of your system easier too).

For developers, the implication is that they need to learn how to write effective unit tests. Beck's experience is that good unit tests:

Run fast (they have short setups, run times, and break downs).

Run in isolation (you should be able to reorder them).

Use data that makes them easy to read and to understand.

Use real data (e.g. copies of production data) when they need to.

Represent one step towards your overall goal.

Continuous Integration in Agile methodology?

Continuous Integration (CI) is a practice in Software Engineering, where all the developers local working code base will be merged to share with a common repository several times during the product development.

It was first adopted as a part of Extreme Programming (XP). The main purpose of Continuous Integration is to prevent developers stepping over each other code and eliminate integration issues.

Continuous Integration works in tandem with other best practices like Configuration management, compilation, software build, deployment, and testing which are bundled into a single automated and repeatable process.

Due to rapid integration of code, it is more likely that defects surface faster than it could compare to normal manual integration.

Continuous Integration has the following built in automation to check the validity of the code that was checked in:

- Static code analysis: Reporting the results of static code execution
- Compile: Generating the executable files by linking the code and compiling after
- Unit test: Writing unit tests, executing them, checking code coverage and reporting the results
- Deploy: Build the code and install it into a test/production environment.
- Integration test: Providing results by executing the integration tests.
- Report (dashboard): Indicating the status of key parameters by posting Red, Green, and Yellow to a publicly visible location.

The cycle of Continuous Integration has been shown below in the figure below

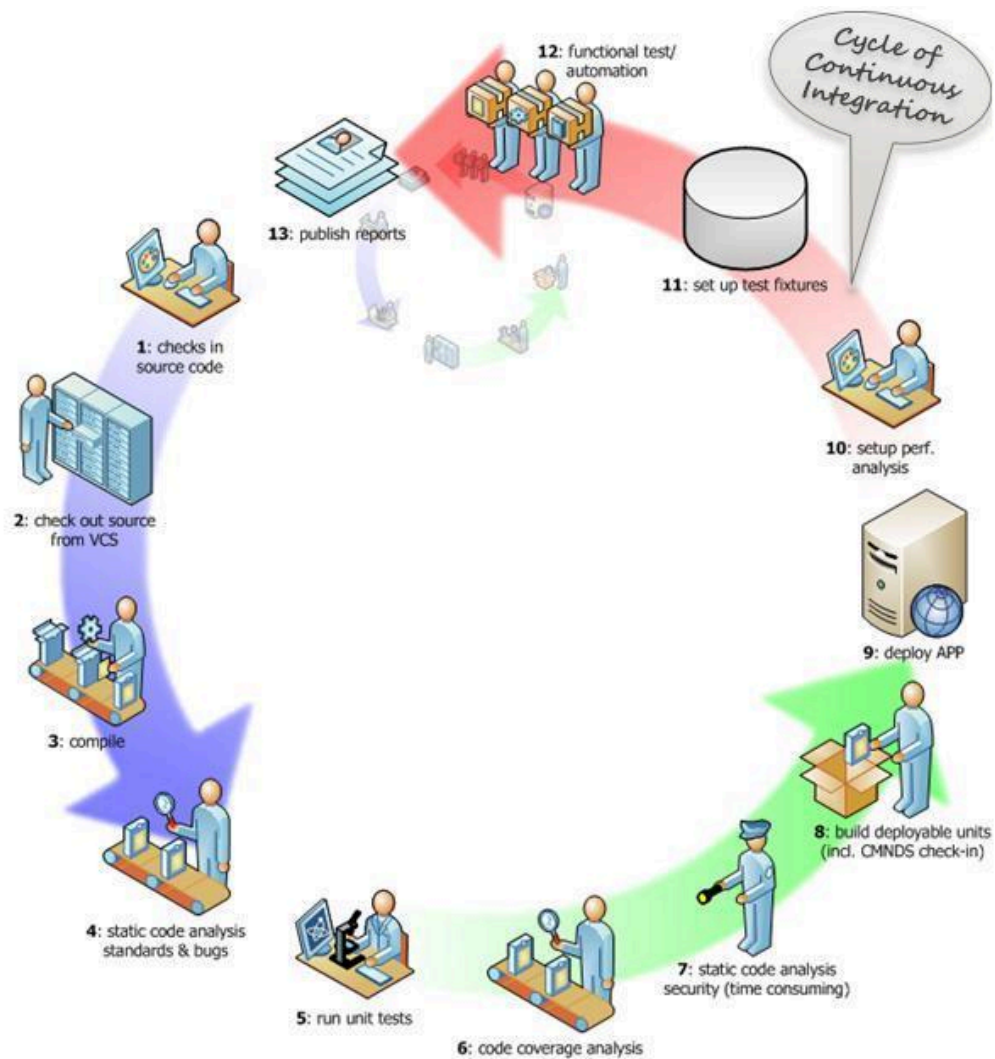


Figure : Cycle of Continuous Integration

- Continuous Integration helps testers to perform automation effectively to uncover defects at a faster rate and improve the regression results.
- The automation will also give the percentage of test coverage area.
- The automation report also covers the number of user stories and functionality mapped to the product increment.
- Automation also reduces the manual testing effort of the testers and makes their life easy.
- Testers can also manually test the areas of failure and collaborate with the developers to fix the defects.
- There are several tools that are being used in organizations as a part of continuous integration to build the automation scripts.
- Few examples of the tools are JUnit, Selenium, SONAR etc.

Continuous Integration automation will reduce the time to deliver, since it replaces the traditional manual testing. Build tools are linked to automation server and build server and deploy to testing, staging and production environments.

Organizations started using these build tools to replace the traditional quality control.

Advantages of Continuous Integration

- Enables a quick feedback mechanism on the build results
- Helps collaboration between various product teams within the same organization
- Decreases the risk of regression since the code is often churned by other developers.
- Maintains version control within for various product releases and patch releases.
- Reduces the technical debt within the code.
- Allows earlier detection and prevention of defects
- Reduces manual testing effort
- Provides a facility of falling back to previous versions in case of any problem to the current build

Risks and challenges of Continuous

- Continuous Integration tools maintenance and their administration have associated costs to it.
- Continuous Integration guidelines need to be well established before starting it.
- Test automation is a rare skill in the market and existing testers may have to be trained on that.
- Full fledged test coverage is required to see the benefits to automation.
- Teams sometimes depend too much on the unit testing and ignore automation and acceptance testing.